Selected Problem Statements

ID1063 - Introduction to Programming
Problem statements with visible test cases

Boundary Sum

Tic-tac-toe Evaluator

Largest Rectangle

Second Largest Distinct Element

1. Boundary Sum

Write a C program that reads a rectangular integer matrix and computes the sum of all elements on its boundary. A boundary element is any element in the first row, last row, first column, or last column.

Each boundary cell must be counted exactly once. In particular, your program must also work correctly when the matrix has only one row, only one column, or consists of a single element.

Input format

The first line contains two integers **rows** and **cols**. The next **rows** lines contain **cols** integers each, giving the matrix row by row.

Constraints

- 1 <= rows, cols <= 100.
- Each matrix entry fits in the C `int` type.

Output format

Print one integer: the sum of the boundary elements. Do not print prompts or explanatory text.

Visible test cases

Test case 1

Input

```
3 4
1 2 3 4
5 6 7 8
9 10 11 12
```

Expected output

```
65
```

Test case 2

Input

```
1 5
2 4 6 8 10
```

Expected output

```
30
```

Test case 3

Input

```
5 1
3
-1
4
2
7
```

Expected output

```
15
```

Test case 4

Input

```
1 1
-5
```

Expected output

```
-5
```

Test case 5

Input

```
2 3
1 2 3
4 5 6
```

Expected output

```
21
```

Test case 6

Input

```
4 4
1 1 1 1
1 9 9 1
1 9 9 1
1 1 1 1
```

Expected output

```
12
```

2. Tic-tac-toe Evaluator

A tic-tac-toe board is represented by a 3 x 3 grid. Each cell contains `X`, `O`, or `-`, where `-` denotes an empty cell.

Evaluate the board. A player wins if that player occupies all three cells in any row, any column, or either diagonal. If neither player has won and no empty cell remains, the result is a draw. Otherwise, the game is still in progress.

You may assume that every input board is a legal position reachable during a normal game, and that both players cannot have winning lines simultaneously.

Input format

The input contains three lines. Each line contains exactly three characters, each of which is `X`, `O`, or `-`. There are no spaces.

Output format

Print exactly one of the following:

- `X WINS` if X has a winning line.
- `O WINS` if O has a winning line.
- `DRAW` if the board is full and neither player has won.
- `IN PROGRESS` otherwise.

Visible test cases

Test case 1

Input

```
XXX
OO-
---
```

Expected output

```
X WINS
```

Test case 2

Input

```
OXX
O-X
O--
```

Expected output

```
O WINS
```

Test case 3

Input

```
XOO
-X-
--X
```

Expected output

```
X WINS
```

Test case 4

Input

```
XXO
XO-
O--
```

Expected output

```
O WINS
```

Test case 5

Input

```
XOX
XXO
OXO
```

Expected output

```
DRAW
```

Test case 6

Input

```
XO-
-X-
O--
```

Expected output

```
IN PROGRESS
```

3. Largest Rectangle

You are given **n** positive integers representing the heights of **n** buildings placed next to each other. For every continuous group of buildings from position **i** to position **j**, consider the rectangle formed by that group.

The width of the rectangle is the number of selected buildings. The height is the smallest building height in that group. Therefore:

```
Area = minimum height x width
```

Find the maximum possible area among all continuous groups of buildings.

Required function

Define and use the following function:

```
int blockArea(int heights[], int i, int j);
```

The function must return the area of the rectangle corresponding to the continuous block from index **i** through index **j**, inclusive. It should find the minimum height in that block and multiply it by the width `j - i + 1`.

In `main`, call `blockArea` for every possible continuous block and print the largest returned value.

Input format

The first line contains an integer **n**. The second line contains **n** positive integers representing the building heights.

Constraints

- 1 <= n <= 1000.
- 1 <= heights[i] <= 10000.

Output format

Print a single integer: the maximum possible rectangle area.

Example

For the input `3 5 2 5 5 4`, the last three buildings have heights 5, 5, and 4. Their minimum height is 4 and their width is 3, so `blockArea(heights, 3, 5)` returns 12.

Visible test cases

Test case 1
Input
6 3 5 2 5 5 4
Expected output
12

Test case 2
Input
1 7
Expected output
7

Test case 3

Input

```
5
1 2 3 4 5
```

Expected output

```
9
```

Test case 4

Input

```
5
5 4 3 2 1
```

Expected output

```
9
```

Test case 5

Input

```
4
6 6 6 6
```

Expected output

```
24
```

Test case 6

Input

```
7
2 1 4 5 1 3 3
```

Expected output

```
8
```

4. Second Largest Distinct Element

Write a C program that reads a single integer array. Call a function named `find_second_largest` that returns the second largest distinct element. If fewer than two distinct values exist, the function must return -1.

What "second largest distinct" means

The answer is the largest value that is strictly smaller than the maximum value. Repeated copies of the maximum do not count as another distinct value.

- In `9 9 8 8 7`, the answer is 8.
- In `5 5 5`, there is only one distinct value, so the answer is -1.
- In `-3 -8 -5`, the answer is -5 because -3 is the largest value.

What you need to do

Read the array size and its elements. Call `find_second_largest` exactly once, passing the array and its size, and print its result. The function should inspect the values in a single pass and keep track of the two best distinct values seen so far. Use only a constant amount of extra space inside the function; do not sort the array.

Use the following function signature:

```
int find_second_largest(const int a[], int n);
```

Input format

The first line contains a positive integer **n**, the number of elements. The next line contains **n** space-separated integers. You may assume that all input values fit in the C `int` type and that the array fits in available memory.

Output format

Print a single integer: the second largest distinct element, or -1 if fewer than two distinct values exist. Do not print an array index or any other text.

Visible test cases

Test case 1
Input
`5` `9 9 8 8 7`
Expected output
`8`

Test case 2
Input
`3` `5 5 5`
Expected output
`-1`

Test case 3

Input

```
3
-3 -8 -5
```

Expected output

```
-5
```

Test case 4

Input

```
1
42
```

Expected output

```
-1
```

Test case 5

Input

```
7
1 3 2 7 7 5 6
```

Expected output

```
6
```

Test case 6

Input

```
5
0 -1 0 -2 -1
```

Expected output

```
-1
```

In test case 6, -1 is the actual second largest distinct value.